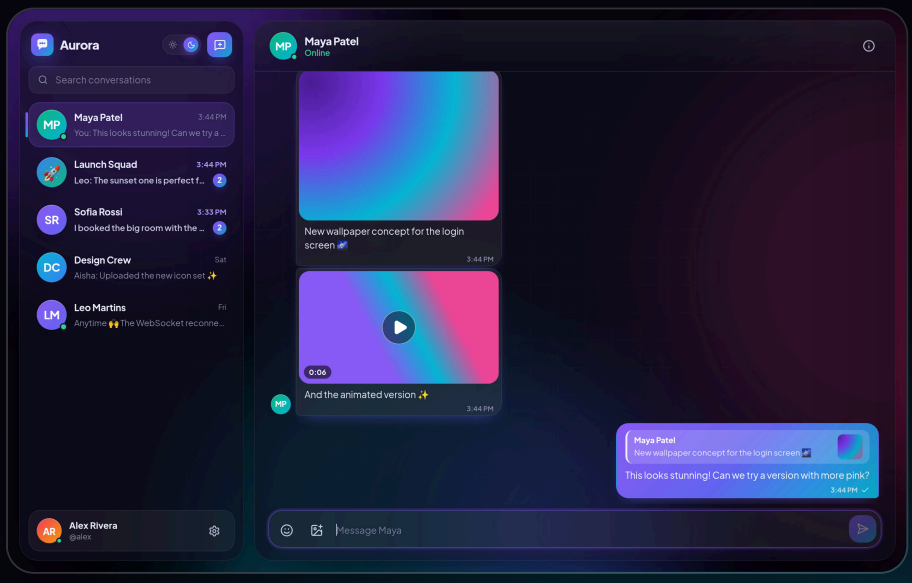
Aurora Chat

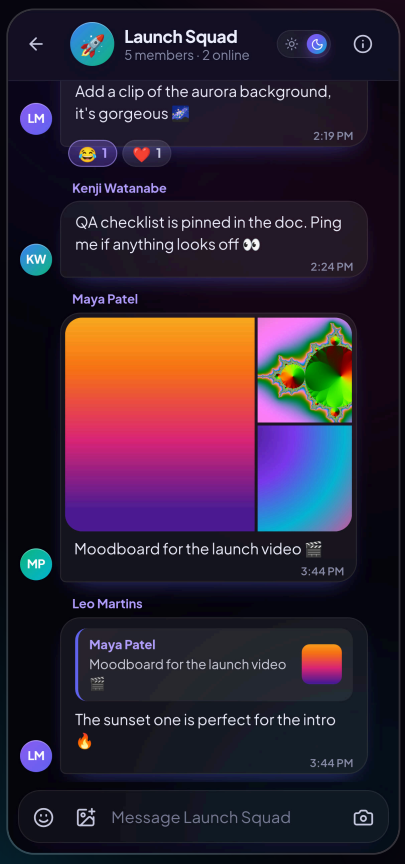
Project guide · v2.1 · upgrade without losing data · downloads · always awake

Real-time chat, now on your phone.

An installable chat app with photo and video sharing, downloads and push notifications, built with a Java Spring Boot backend and a React frontend. Version 2.1 upgrades your existing GitHub repo and database without losing any data, and keeps the free server awake with an automatic ping every 10 minutes.

Spring Boot 3.5 · Java 17+ | React 19 · Vite 8 · PWA | Web Push (VAPID) | Downloads | Keep-alive every 10 min | Vercel + Render + Neon





Contents

01 What's new in v2 & v2.1 — mobile · notifications · media · downloads · keep-awake · safe upgrade

02 App tour (phone + desktop, light & dark) — screenshots

03 Architecture — media flow · push flow · data model · security

04 Project overview (README) — stack, structure, API, configuration

05 Full guide: upgrade, keep-awake, new install — Part B · Part C · Part A

06 Verification results — 26 tests · 40 browser checks · upgrade test

What's new in v2

📱 Built for phones

- Installable app (PWA) with an icon, splash screen and offline shell
- Long-press for actions, swipe right to reply, double-tap ❤️
- Composer stays above the keyboard; notch & safe-area aware
- Bottom sheets, 44 px touch targets, back button navigation
- Faster: memoized message rows, lighter effects, code splitting

🔔 Notifications

- Push notifications even when the app is closed
- Messages from one chat stack into one notification; tap opens that chat
- In-app banners + soft sound for other chats
- Unread badge on the app icon; mute any chat
- Works on Android, desktop, and iPhone (after Add to Home Screen)

📷 Photos & videos

- Gallery, camera, paste or drag & drop
- Compressed on the device, upload progress, blurred previews
- Album grids, video posters & durations
- Full-screen viewer: swipe, pinch-zoom, share, save
- Shared-media gallery in each chat's info panel

New in v2.1

🔄 Keep your data

- Upgrade in your **existing GitHub repo** (one script or GitHub Desktop)
- Same Render service, Vercel project and Neon database
- Additive database changes only; everyone stays signed in
- Backup + rollback steps, rollback tested

⬇️ Downloads

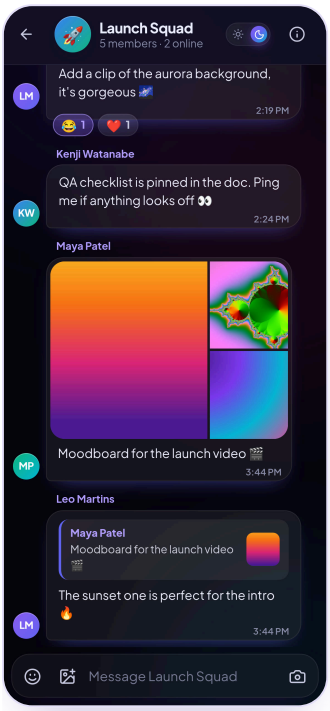
- Hover → ⬇ on any photo or video (computer)
- ⋯ → **Download all** for albums
- Phone: long-press → **Save photo / Save all** → Photos/Gallery
- Works for Cloudinary and local storage

☕ Always awake

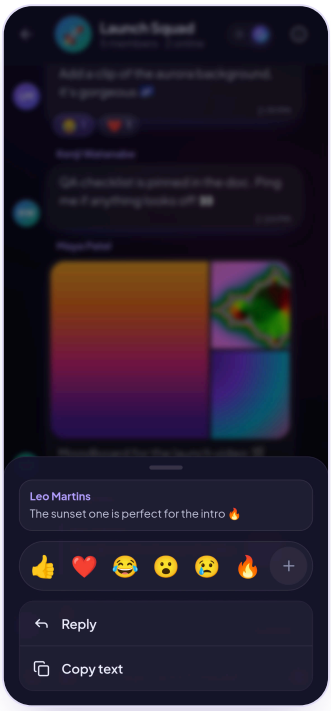
- Server pings itself every 10 minutes (automatic on Render)
- GitHub Actions schedule event every 10 minutes
- Version, uptime and keep-awake status in Settings → About

Both README files are updated in the project: `README.md` (GitHub page) and `SETUP_AND_DEPLOYMENT.md` (the full guide: **Part B** upgrade without losing data, **Part C** keep-awake, **Part A** new installation). Both are reproduced in this PDF.

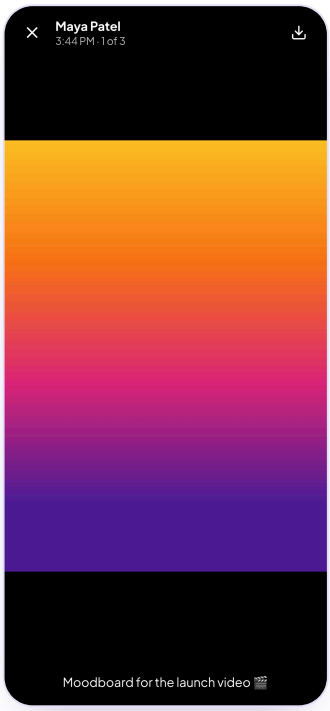
1 · App tour: phone



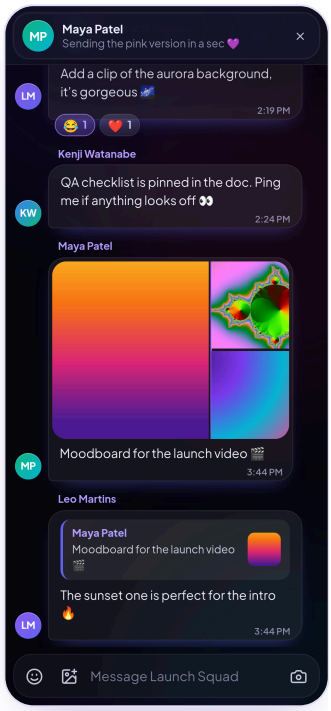
Group chat with an album, reply with a photo quote



Long-press: quick reactions + actions



Full-screen viewer: swipe, zoom, save



In-app notification for another chat

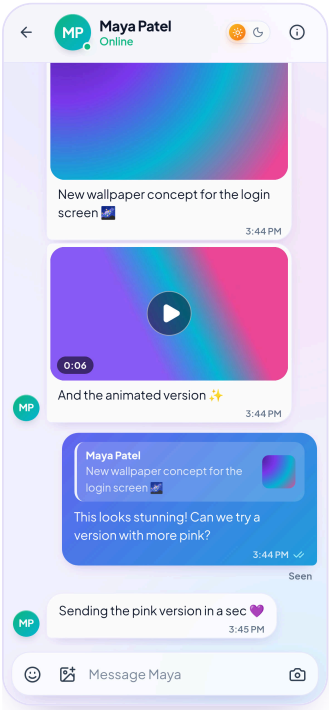
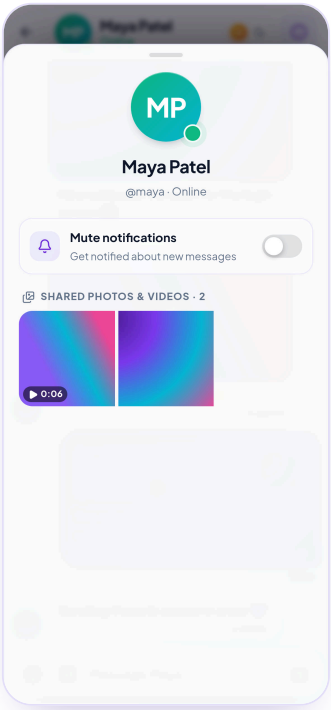
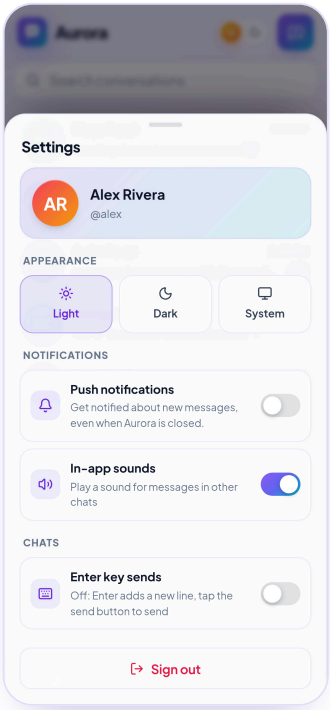


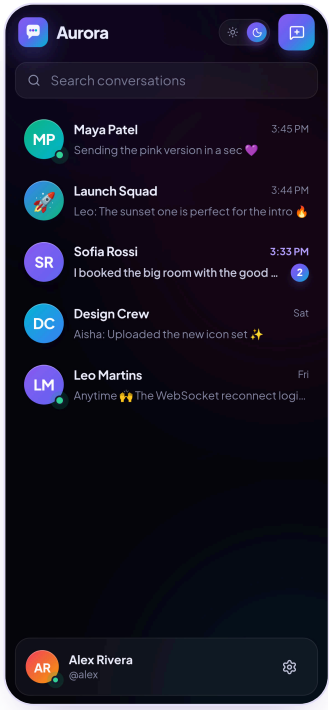
Photo & video messages, read receipts



Chat info: mute + shared photos & videos

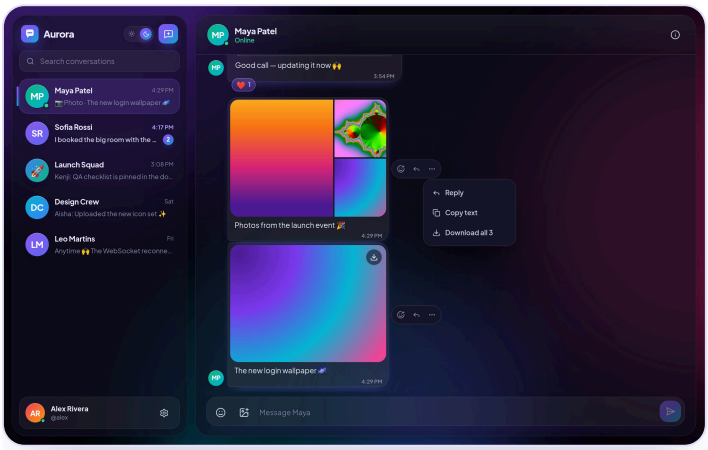


Settings: theme, notifications, sounds

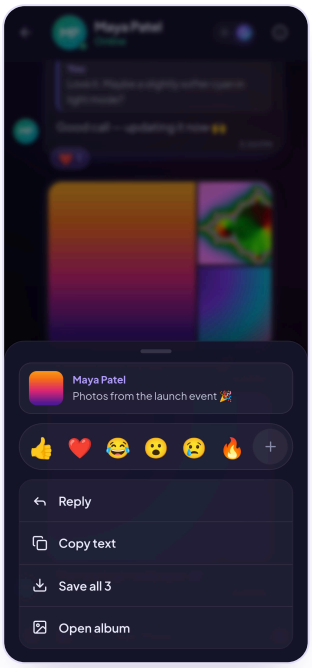


Conversation list with unread badges

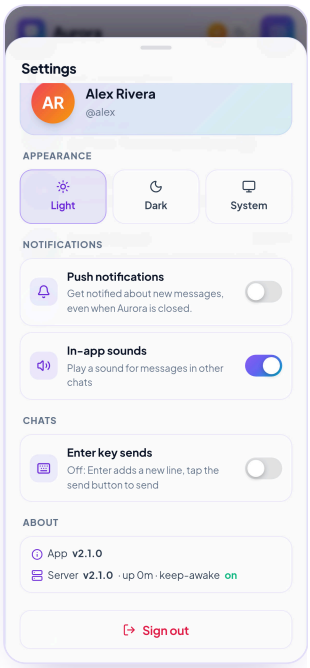
New in v2.1: downloads & About



Hover → ⬇ downloads one photo; ⋯ → Download all saves the album

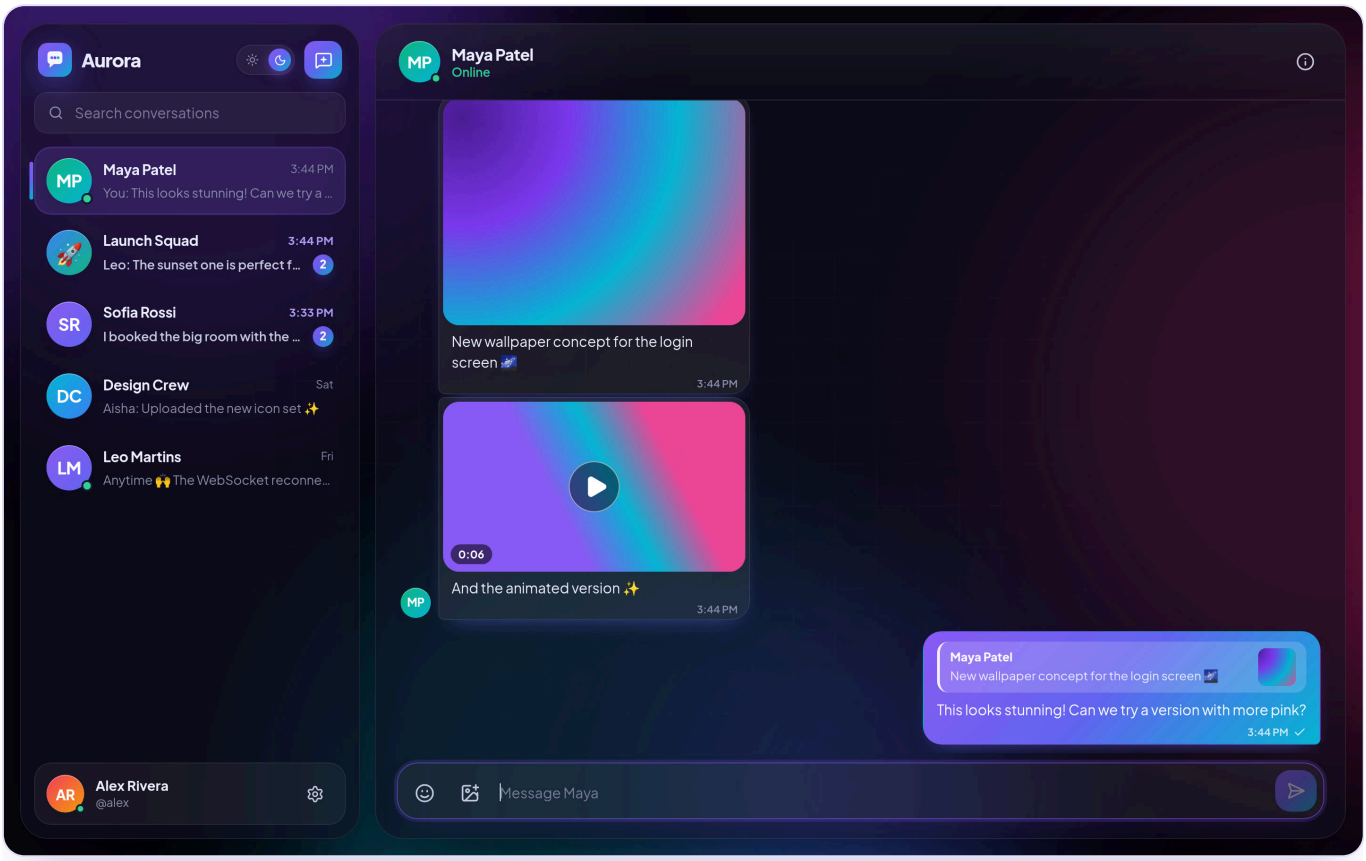


Phone: long-press → Save all

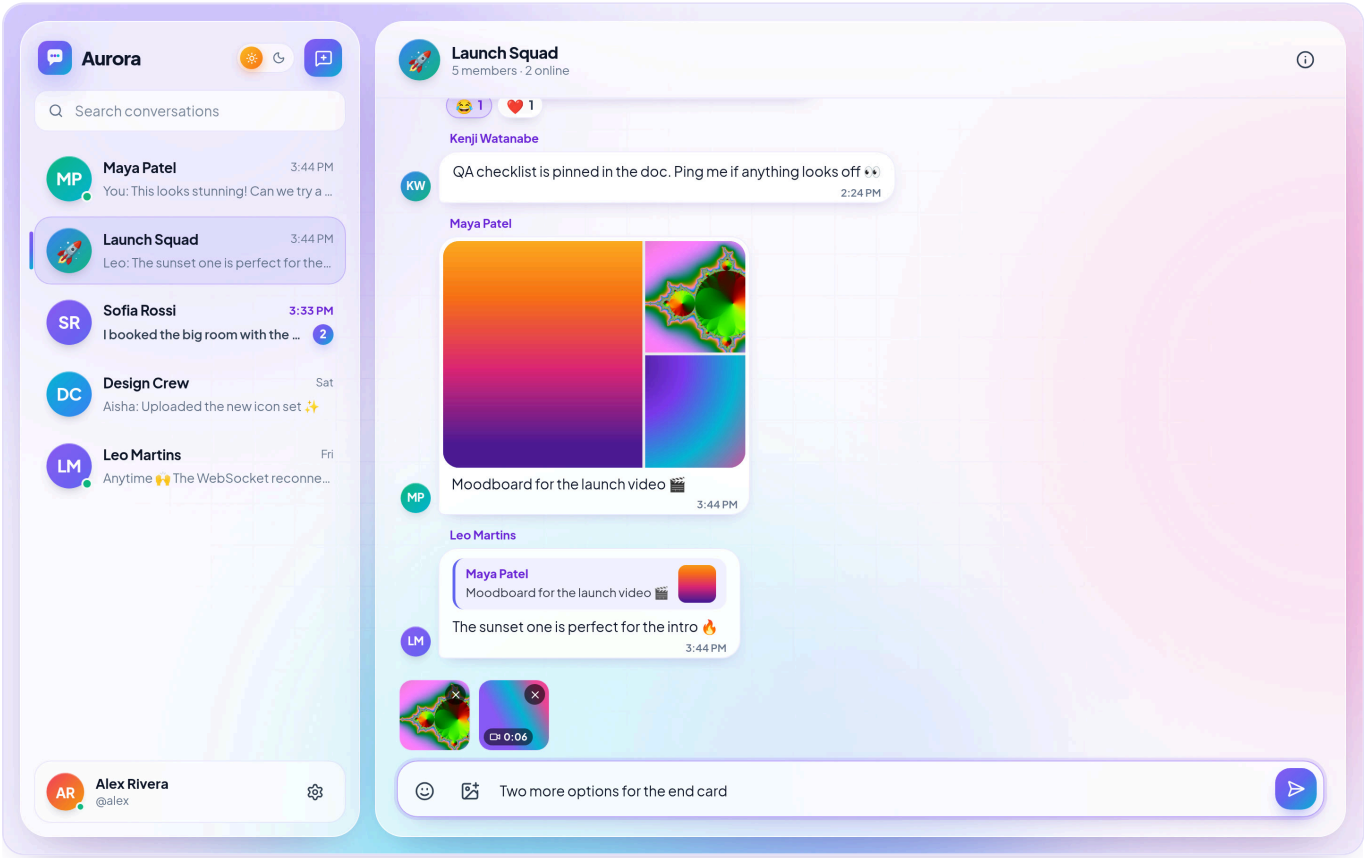


Settings → About: versions + keep-awake

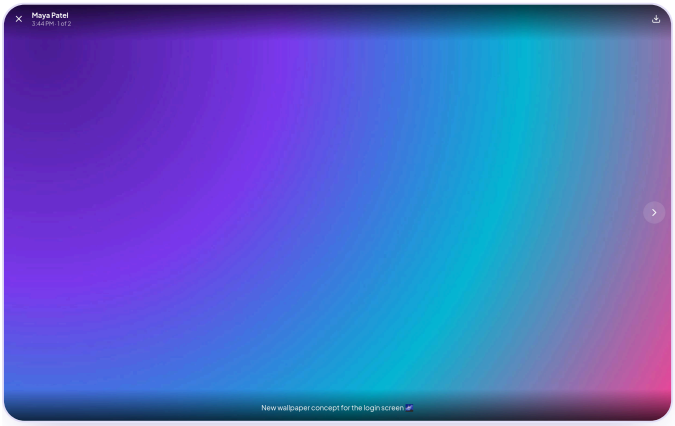
App tour: desktop



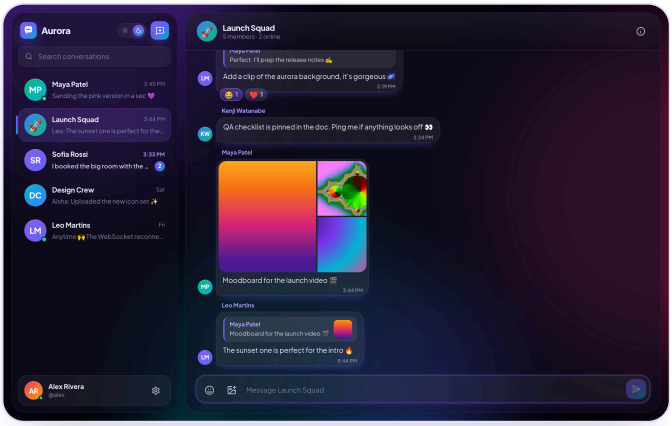
Dark mode: photo and video messages, reply with a thumbnail, "Seen" receipt



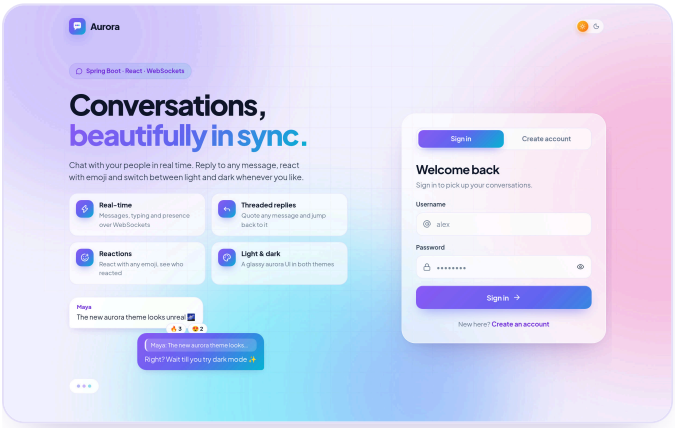
Light mode: album grid, reply quotes, attachments ready to send with a caption



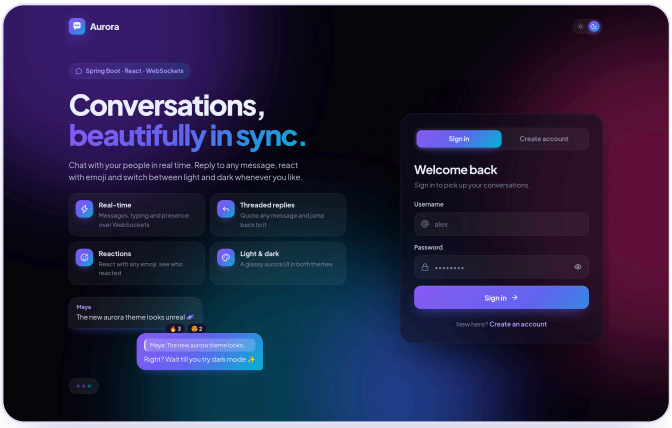
Desktop viewer (arrow keys, double-click to zoom)



Group chat in dark mode



Sign in / create account (light)



Sign in / create account (dark)

Tips: hover a message (or long-press on a phone) for react · reply · more. Double-click/double-tap to ❤️. Paste a screenshot or drop files to share them. **Enter** sends on desktop; on phones Enter adds a line (configurable in Settings).

2 · Architecture

VERCEL

React PWA

- React 19 · TypeScript · Vite 8
- STOMP client, optimistic UI
- Media processing on device

SERVICE WORKER

sw.js

- Offline app shell
- Push → notification
- Click → open the chat

RENDER · DOCKER

Spring Boot API

- REST `/api/**` + STOMP `/ws`
- JWT security for HTTP and WebSocket
- **media**: sniffing, limits, storage, range streaming, clean-up
- **push**: VAPID keys, RFC 8291 encryption, fan-out, expiry handling
- Events published after commit

NEON

PostgreSQL

- users, chats, messages
- attachments, subscriptions

CLOUDINARY

Media CDN

- photos, videos, posters
- signed REST uploads

PUSH SERVICES

FCM · Mozilla · Apple · WNS

- deliver encrypted messages

Sending a photo

1. **Pick** a photo or video, or paste or drop it. The app resizes and compresses photos (max 2048 px), grabs a video poster frame, and makes a tiny blurred preview.
2. **Upload** starts right away (`POST /api/media`) with a progress bar. The server checks the real file type from its bytes, enforces size limits, and stores the file on Cloudinary or disk.
3. **Send** shows the message instantly using the local copy. When the uploads finish, the message is posted with its `attachmentIds` .
4. **Deliver**: members receive a `MESSAGE_CREATED` event and see blurred previews that sharpen as the images load.

How a notification reaches a phone

1. The browser subscribes with the server's VAPID public key. Its endpoint and keys are saved (only real push-service hosts are accepted).
2. After a message commits, a background thread builds a payload for each member who isn't muted: title, preview, chat link, and unread count for the badge.
3. The payload is encrypted for that one browser (ECDH P-256 + AES-128-GCM, RFC 8291), signed with an ES256 VAPID JWT, and posted to the browser's push service.
4. The service worker shows the notification, stacking messages from the same chat. If the app is open and focused it shows nothing, because the app shows its own banner. Tapping the notification opens the chat.

Data model

app_users

`id · username · display_name`
`password_hash · last_seen_at`

conversations

`id · type · name`
`direct_key · updated_at`

conversation_members

`conversation · user`
`last_read_message_id ·`
`muted`

messages · reactions

`content · reply_to ·`
`edited/deleted`
`message · user · emoji`

attachments (new)

`id (uuid) · message ·`
`uploader · kind`
`mime · size · w×h · duration`
`url · thumbnail ·`
`placeholder`

push_subscriptions (new)

`user · endpoint (unique)`
`p256dh · auth · user_agent`

app_settings (new)

`setting_key · setting_value`
`(server VAPID key pair)`

Security highlights

- Uploads are checked by their **bytes**, not their names, so SVG, HTML or scripts disguised as photos are rejected. Local files are served with `nosniff` and a sandbox CSP.
- Media URLs use unguessable 122-bit random ids (needed because image and video tags can't send auth headers). Deleting a message deletes its files.
- Push subscriptions only accept real push-service hosts, so the server can't be tricked into calling internal URLs. Notification contents are end-to-end encrypted to each device.
- JWT on REST and on STOMP `CONNECT`; every chat, message, media and mute request checks membership.

3 Project overview

From README.md, the GitHub project page

✨ Features

Feature	What it does
⚡ **Real-time messaging**	Messages appear instantly for everyone in the chat (STOMP over WebSocket). The app reconnects and catches up automatically, including when a phone wakes the app.
📷 **Photos & videos**	Send up to 10 at once from the gallery, camera, clipboard paste or drag & drop. Photos are compressed on the device (a 3.5 MB photo became about 1.5 MB in testing). Videos get a poster frame. You see upload progress, a blurred preview while files load, and albums laid out in a grid.
⬇️ **Download received media**	On a computer, hover a photo or video and click ⬇, or use ⋯ → **Download all** for an album. On a phone, long-press → **Save photo / Save all**, which opens the share sheet → *Save Image* (Photos/Gallery). Files get neutral names like `aurora-20261026-153812-1a2b.jpg`.
🖼️ **Full-screen viewer**	Swipe between photos and videos, pinch or double-tap to zoom, swipe down to close, and share or save files.
🔔 **Push notifications**	Get alerted about new messages even when the app is closed (Web Push with VAPID, implemented with the plain JDK). Several messages from one chat stack into one notification. Tapping a notification opens that chat. The home-screen icon shows an unread badge.
🔕 **Mute & in-app alerts**	Mute any chat. While the app is open, messages in other chats show a tappable banner with a soft sound.
📱 **Mobile-first, installable**	Install it as an app (PWA) with an offline app shell. On phones: long-press a message for actions, swipe right to reply, double-tap to ❤️. The composer stays above the keyboard, the layout respects notches and safe areas, and the back button works.
↩ **Replies**	Reply to any message, photos included (the quote shows a thumbnail). Tap the quote to jump to the original.
😍 **Reactions**	Quick reactions, a searchable emoji picker, live counts, and you can see who reacted.
👥 **Direct & group chats**	1:1 and named group chats, each with an info panel listing members and every shared photo and video.
🟢 **Presence, typing, receipts**	Online dots, "last seen" times, typing indicators, ✓ sent / ✓✓ seen, and unread badges.
🌓 **Light & dark mode**	Light, dark, or follow the system, switched with a circular reveal animation.
☕ **Stays awake on free hosting**	The server pings its own public address every 10 minutes (automatic on Render), and a scheduled GitHub Actions event every 10 minutes is included as a second layer, so the free instance never falls asleep. `/api/health` shows the version, uptime and keep-alive status.
🔄 **Safe upgrades**	Database changes are additive only. Upgrading keeps every user, chat and message (tested on PostgreSQL 17 and H2), and `scripts/update-repo.sh` puts a new version into your existing GitHub repo.

Feature	What it does
🔐 **Secure by default**	BCrypt passwords, JWT for REST and WebSocket, per-chat authorization. Uploads are type-checked by their actual bytes, push endpoints are restricted to an allowlist, and notifications are end-to-end encrypted.

🧱 Tech stack

Layer	Technology
Frontend	React 19, TypeScript, Vite 8, `@stomp/stompjs`, lucide icons, hand-written CSS design system, service worker (offline shell + push), Web App Manifest
Backend	Java 17+, Spring Boot 3.5 (Web, WebSocket/STOMP, Security, Data JPA, Validation), JJWT; Web Push (RFC 8291/8292) implemented with the plain JDK
Storage	PostgreSQL (e.g. free Neon) or zero-setup H2 locally · media on **Cloudinary** (free) or local disk
Hosting (free)	Frontend on **Vercel** · API on **Render** (Docker) · Postgres on **Neon** · media on **Cloudinary**

🚀 Quick start (local)

Prerequisites: JDK 17+ and Node.js 20.19+ (or 22+). No database or cloud accounts needed.

```
git clone https://github.com/<your-username>/aurora-chat.git
cd aurora-chat

# 1) Backend → http://localhost:8080   (demo users are optional)
cd backend
APP_DEMO_DATA_ENABLED=true ./mvnw spring-boot:run        # Windows CMD: set "APP_DEMO_DATA_ENABLED=true"
&& mvnw.cmd spring-boot:run

# 2) Frontend → http://localhost:5173   (in a second terminal)
cd frontend
cp .env.example .env.local
npm install
npm run dev
```

Open http://localhost:5173 and sign in as **alex / password123** (demo data), or create an account. To chat with yourself, open a **private window** and sign in as **maya / password123**. Photos and videos are saved in `backend/data/uploads`. Notifications work on `localhost` in Chrome, Edge and Firefox: turn them on under ⚙️ Settings.

> Full step-by-step instructions, including free deployment to Vercel, are in **SETUP_AND_DEPLOYMENT.md**.

🔄 Upgrading an existing installation

Already running an older Aurora Chat? Keep the **same GitHub repo, Render service, Vercel project and Neon database**. All data stays:

```
git clone https://github.com/<you>/<your-repo>.git my-aurora-repo   # your existing repo
cd aurora-chat                                                      # the new version (unzipped)
bash scripts/update-repo.sh ../my-aurora-repo                       # copies it in, asks to commit + push
```

Render and Vercel redeploy automatically. Then add `CLOUDINARY_URL` on Render. Backup, GitHub Desktop and rollback steps: **SETUP_AND_DEPLOYMENT.md → Part B**.

📁 Project structure

```
aurora-chat/
├── backend/                          Spring Boot API (Maven)
│   ├── src/main/java/com/aurora/chat/
│   │   ├── auth/                     register / login / me
│   │   ├── chat/                     conversations, messages, reactions, typing, mute, events
│   │   ├── media/                    uploads, type sniffing, local + Cloudinary storage, streaming, clean-
up
│   │   ├── push/                     Web Push: VAPID keys, RFC 8291 encryption, subscriptions,
notifications
│   │   ├── user/                     users, presence
│   │   ├── security/                 JWT, HTTP filter, STOMP auth interceptor
│   │   ├── config/                   security, WebSocket, async, DATABASE_URL parsing, demo data
│   │   ├── common/ · dto/            mapping, errors, health, request/response records
│   ├── src/test/java/...             26 tests: REST, WebSocket, media, downloads, push, Cloudinary, keep-
alive
│   └── Dockerfile
├── frontend/                         React + Vite PWA
│   ├── public/                       sw.js (service worker), manifest.webmanifest, icons/
│   ├── src/components/               chat UI, media grid & viewer, sheets, settings, banners…
│   ├── src/context/                  auth, theme, toasts, chat state (reducer + STOMP client)
│   ├── src/lib/                      API, media processing & uploads, push, PWA, sounds, formatting
│   └── vercel.json
├── scripts/update-repo.sh            puts a new version into your existing Git repo (keeps data & history)
├── .github/workflows/                ci.yml (build + test) · keep-alive.yml (pings the backend every 10
min)
├── docs/                             PDF guide + screenshots
├── render.yaml                       one-click Render blueprint for the API
├── docker-compose.yml                optional local Postgres + API
├── README.md                         ← you are here
└── SETUP_AND_DEPLOYMENT.md           setup & free hosting guide
```

🔌 API reference

All endpoints except register/login/health and media downloads require `Authorization: Bearer <token>`. Errors look like `{"status":400,"error":"Bad Request","message":"…"}`.

Method	Endpoint	Description
`POST`	`/api/auth/register` · `/api/auth/login`	→ `{token, user}`
`GET`	`/api/auth/me`	Current user
`GET`	`/api/users?query=`	Search people (excludes you)
`GET`	`/api/conversations`	Your chats with last message, unread count, read states, `muted`

Method	Endpoint	Description
`POST`	`/api/conversations/direct` · `/group`	`{userId}` · `{name, memberIds[]}`
`GET`	`/api/conversations/{id}/messages?before=&limit=`	Page of messages (oldest → newest)
`POST`	`/api/conversations/{id}/messages`	`{content?, replyToId?, clientId?, attachmentIds?[]}` (text and/or up to 10 uploads)
`POST`	`/api/conversations/{id}/read`	`{messageId}` → move your read marker
`PUT`	`/api/conversations/{id}/mute`	`{muted}` → mute/unmute notifications
`GET`	`/api/conversations/{id}/media?before=`	Shared photos & videos, newest first
`PATCH` · `DELETE`	`/api/messages/{id}`	Edit (caption may be empty for media) · delete your message (files are deleted too)
`POST`	`/api/messages/{id}/reactions`	`{emoji}` → toggle your reaction
`POST`	`/api/media`	Multipart upload: `file` (+ optional `poster`, `width`, `height`, `durationMs`, `placeholder`) → attachment
`GET`	`/api/media/{id}` · `/api/media/{id}/thumbnail`	File stream with HTTP range support (public, unguessable id); add `?download=true` to force a download
`GET`	`/api/push/config`	`{enabled, publicKey}` (VAPID)
`POST` · `DELETE`	`/api/push/subscriptions`	Register / remove this browser (`PushSubscription.toJSON()`)
`POST`	`/api/push/test`	Send yourself a test notification
`GET`	`/api/health`	`{status, version, uptime, keepAlive}`: public, no database access

WebSocket (STOMP)

Connect to `ws(s)://<api-host>/ws` with the STOMP header `Authorization: Bearer <token>`.

Direction	Destination	Payload
subscribe	`/user/queue/events`	`{type, conversationId, payload}`: `MESSAGE_CREATED`, `MESSAGE_UPDATED`, `REACTIONS_UPDATED`, `CONVERSATION_UPSERTED`, `TYPING`, `READ`, `MUTE_UPDATED`
subscribe	`/topic/presence`	`{userId, online, lastSeenAt}`
send	`/app/conversations/{id}/typing`	`{typing: true | false}`

⚙️ Configuration

Backend (environment variables)

Variable	Default	Purpose
`PORT`	`8080`	HTTP port (set automatically by most hosts)
`DATABASE_URL`	*(unset → H2 file DB)*	Postgres URL, e.g. `postgresql://user:pass@host/db?sslmode=require` (Neon/Render format is fine)
`APP_JWT_SECRET`	dev value	**Required in production.** Any long random string
`APP_JWT_EXPIRATION`	`7d`	Token lifetime
`APP_CORS_ALLOWED_ORIGINS`	`http://localhost:5173,…`	Comma-separated frontend origins; patterns like `https://*.vercel.app` allowed
`CLOUDINARY_URL`	*(unset → local disk)*	`cloudinary://API_KEY:API_SECRET@CLOUD_NAME` from the Cloudinary dashboard. **Needed on hosts with temporary disks (Render free).**
`APP_MEDIA_STORAGE`	`auto`	`auto`, `local` or `cloudinary`
`APP_MEDIA_DIR`	`./data/uploads`	Folder for local media storage
`APP_MEDIA_MAX_IMAGE_MB` · `APP_MEDIA_MAX_VIDEO_MB`	`10` · `50`	Upload limits per file
`APP_PUSH_SUBJECT`	`mailto:notifications@aurora-chat.dev`	Your contact `mailto:` or `https:` URL for push services (Apple requires a valid one)
`APP_PUSH_VAPID_PUBLIC_KEY` · `…_PRIVATE_KEY`	*(generated)*	Optional; otherwise generated once and stored in the database
`APP_PUSH_ENABLED`	`true`	Turn push notifications off entirely
`APP_PUBLIC_URL`	*(Render: automatic)*	Public API URL; lets notifications show photo previews when media is on local disk
`APP_KEEP_ALIVE_ENABLED`	`true`	Ping `<public URL>/api/health` so free hosts don't put the server to sleep
`APP_KEEP_ALIVE_URL`	*(Render: automatic)*	URL to ping (defaults to `APP_PUBLIC_URL`, or `RENDER_EXTERNAL_URL` on Render)
`APP_KEEP_ALIVE_INTERVAL`	`10m`	How often to ping (keep it under Render's 15-minute sleep timer)
`APP_DEMO_DATA_ENABLED`	`false`	Seed demo users/conversations into an **empty** database

Frontend (build-time: `frontend/.env.local` locally, or Vercel → Environment Variables)

Variable	Example	Purpose
`VITE_API_URL`	`https://aurora-chat-api.onrender.com`	Backend base URL (no trailing slash)
`VITE_WS_URL`	*(optional)*	Override the WebSocket URL (defaults to `VITE_API_URL` + `/ws`)

📱 Notifications & install, by platform

Platform	Install	Push notifications
Android (Chrome, Edge, Samsung)	Menu → **Install app** (or the in-app Settings button)	✅ In the browser or installed
iPhone / iPad (iOS 16.4+)	Safari → Share → **Add to Home Screen**	✅ After installing to the Home Screen (Apple requirement)
Desktop (Chrome, Edge, Firefox, Safari 16+)	Install icon in the address bar (Chromium)	✅

🧪 Testing

```
cd backend && ./mvnw test          # 26 tests
cd frontend && npm run build       # strict TypeScript + production build
```

The backend tests cover REST flows and WebSocket delivery. They also cover media: upload, type sniffing, range streaming, deleting files with their message, and cleaning up uploads that were never sent. On the push side, they check Cloudinary request signing (against Cloudinary's documented example), VAPID, and a full delivery path to a fake push service. One test asserts that the Web Push encryption matches a vector from Node's `http_ece` (the library behind the `web-push` npm package) byte for byte.

The app was also checked end to end with real browsers: an emulated phone and a laptop (17 mobile/media/notification checks, 7 download checks and 16 regression checks). The upgrade path was run on PostgreSQL 17 and H2: data from the previous version survived upgrading, rolling back and upgrading again.

🛣️ Ideas for next steps

- Voice messages and file attachments (PDFs, docs)
- Message search and pinned messages
- Group management (rename, add/remove members, admins)
- Horizontal scaling with an external STOMP broker (RabbitMQ) and Redis-backed presence

4 Full guide: upgrade, keep-awake, new install

From SETUP_AND_DEPLOYMENT.md: Part B keeps your data in the same GitHub repo · Part C keeps the app awake · Part A fresh setup

Everything you need to run Aurora Chat for **$0**: a live, installable chat app with photo and video sharing, downloads and push notifications, which stays awake around the clock.

Your situation	Follow
🆕 **First time**: nothing deployed yet	**Part A**, steps 1–12
🔄 **Already running Aurora Chat** and want the new version **without losing data, in the same GitHub repo**	Part B — Upgrade
💤 Stop the free server from **falling asleep**	Part C — Keep the app awake

Part A — New installation

1. How the free hosting fits together
2. Prerequisites
3. Run it locally
4. Push the code to GitHub
5. Create a free PostgreSQL database (Neon)
6. Create free media storage (Cloudinary)
7. Deploy the Java backend (Render)
8. Deploy the React frontend (Vercel)
9. Connect frontend and backend (CORS)
10. Test your live app, install it on your phone, turn on notifications
11. Optional extras
12. Troubleshooting

Part B — Upgrade an existing installation (keep data, same repo) · **Part C — Keep the app awake**

Expect about **40 minutes** for a first deployment, and **15 minutes** for an upgrade.

Part B — Upgrade your existing app (keep all data, same GitHub repo)

Follow this part if Aurora Chat is **already running** (your own GitHub repo connected to Render and Vercel). Don't follow Part A for this: you don't need a new repository, database, Render service or Vercel project. You update the code in your existing repo, and Render and Vercel redeploy it by themselves.

B.1 What happens to your data

Your data	After the upgrade
Accounts, passwords, chats, messages, replies, reactions, read receipts	**Kept exactly as they are.** The new version only *adds* things to the database (3 new tables and 1 new optional column). It never changes or deletes existing rows.

Your data	After the upgrade
Who is signed in	Everyone stays signed in, as long as `APP_JWT_SECRET` on Render stays the same (don't change it).
Your Render / Vercel / Neon settings	Unchanged. New settings are optional, see B.5.

This was tested on **PostgreSQL 17 (the same database as Neon)** and on the local H2 database. Data created with the old version was identical after upgrading, after rolling back to the old version, and after upgrading again.

> ⚠️ If your old deployment had **no** `DATABASE_URL` on Render, the data was in a temporary file that Render wipes on every restart, so there's nothing permanent to keep. Do Part A step 5 (Neon) now so data is kept from here on.

B.2 Make a backup first (2 minutes, recommended)

Neon (your live database). Open https://console.neon.tech → your project → **Branches** → **Create branch**. Name it `backup-before-upgrade`, choose your main branch as the parent and **current point in time**, then click **Create**. This makes an instant copy of your whole database that the app doesn't use. If anything goes wrong you still have every row. You can delete the branch after a few days.

(Alternative, if you have PostgreSQL tools installed: `pg_dump "<your DATABASE_URL>" > aurora-backup.sql` *)*

Local copy (if you also ran it on your computer). Copy the folder `backend/data/` somewhere safe. It holds the local database and uploads.

B.3 Put the new version into your GitHub repo

Pick **one** of the three ways below. All of them keep your repo's history, so you're adding one new commit.

Way 1 — automatic script (macOS, Linux, or Windows with Git Bash)

```
# 1. Get a fresh copy of YOUR repo (any empty folder works)
git clone https://github.com/<your-username>/<your-repo>.git my-aurora-repo

# 2. Unzip the new version next to it (creates the folder aurora-chat/), then:
cd aurora-chat
bash scripts/update-repo.sh ../my-aurora-repo
```

The script copies the new files into your repo and removes 5 old screenshots that are no longer used. It never touches your Git history, local data (`backend/data`), `.env` files or any file you added yourself. It shows a summary and asks **"Commit and push to GitHub now?"**. Answer `y` and you're done: Render and Vercel start redeploying.

> On Windows, use **Git Bash** (installed with Git for Windows): right-click the `aurora-chat` folder → **Open Git Bash here**, then run the `bash scripts/update-repo.sh …` line.

Way 2 — GitHub Desktop (no command line)

1. Install GitHub Desktop and sign in.
2. **File → Clone repository** → choose your Aurora Chat repo → **Clone**.
3. **Repository → Show in Explorer/Finder** to open the cloned folder.

4. Delete everything in that folder **except the hidden `.git` folder**. A fresh clone contains no data, so this is safe. *(To see hidden files: Windows Explorer → View → Show → Hidden items; macOS Finder → `Cmd + Shift + .`)*
5. Unzip the new version and copy **everything inside** its `aurora-chat` folder into the cloned folder.
6. Back in GitHub Desktop you'll see the changes. Type the summary `Update Aurora Chat to v2.1` → **Commit to main** → **Push origin**.

Way 3 — plain Git commands

```
git clone https://github.com/<your-username>/<your-repo>.git my-aurora-repo
cd my-aurora-repo
git rm -r -q .                      # removes the old files from the working copy (history is kept)
cp -R ../aurora-chat/. .            # copy the new version in (unzip it next to the clone first)
git add -A
git commit -m "Update Aurora Chat to v2.1"
git push
```

Ways 2 and 3 replace every file in the repo. If you committed files of your own (notes, images…), copy them back before committing, or use Way 1, which keeps them.

Did you change the code yourself? The update overwrites Aurora Chat's files. Before pushing, run `git diff --cached` (or look at the changes in GitHub Desktop) and re-apply your edits.

Is your project inside a sub-folder of the repo (for example `my-repo/aurora-chat/backend`)? Use that sub-folder everywhere above. Your Render "Root Directory" and Vercel "Root Directory" already point there.

B.4 Watch the automatic redeploys (5–10 minutes)

- **Render** → your service → **Events**: a new deploy starts from your commit. In **Logs** you'll see `Media storage: …`, `Keep-alive: pinging https://….onrender.com/api/health every 10 min` and `Started AuroraChatApplication`.
- **Vercel** → your project → **Deployments**: the newest one becomes **Ready**.
- Open `https://<your-service>.onrender.com/api/health`. It should show `"version":"2.1.0"` and `"keepAlive":"on"`.

Using a Render **Blueprint**? New settings in `render.yaml` aren't filled in automatically for an existing service. Add them in the dashboard (next step). Your existing values, including the generated `APP_JWT_SECRET`, stay as they are.

B.5 Add the new settings on Render (Environment tab)

Key	Needed?	Value
`CLOUDINARY_URL`	**Yes, before anyone shares photos or videos**	From Part A step 6 (`cloudinary://…`). Without it, photos are stored on Render's temporary disk and disappear after a restart.
`APP_PUSH_SUBJECT`	Recommended	`mailto:you@example.com` (contact address for push services)
`APP_KEEP_ALIVE_ENABLED`	No	Keep-alive is **on automatically** on Render. Set it to `false` only to turn it off.

Leave `DATABASE_URL`, `APP_JWT_SECRET` and `APP_CORS_ALLOWED_ORIGINS` **exactly as they are**. Saving the new settings triggers one more short redeploy.

B.6 Check that everything is there

1. Open your app (same Vercel address as before). You should still be signed in.
2. Your old chats and messages are all there. ⚙️ **Settings → About** shows **App v2.1.0** and **Server v2.1.0 · keep-awake on**.
3. Send a photo, turn on notifications (Part A step 10.3), and set up the second keep-awake layer (Part C).

B.7 If something goes wrong: roll back

Your data is safe either way. The old version runs fine on the upgraded database (tested).

- **Fastest:** Render → service → **Events** → pick the previous successful deploy → **Rollback** (if your plan shows it). Vercel → **Deployments** → previous deployment → ⋯ → **Instant Rollback**.
- **Always works:** undo the commit on GitHub, which redeploys the old code on both:

```
cd my-aurora-repo && git revert --no-edit HEAD && git push
```

- **Data problem?** Neon → **Branches** → `backup-before-upgrade` → **Restore** (or point `DATABASE_URL` at the backup branch's connection string).

Photo messages sent with the new version show up as text-only in the old version, and come back with their photos when you upgrade again.

Part C — Keep the app awake (automatic, every 10 minutes)

Render's free plan puts your backend to sleep after **15 minutes without visitors**. The next visitor then waits about a minute, and push notifications can't be sent while it sleeps. Aurora Chat prevents this with **two automatic layers**. Use both.

C.1 Layer 1: built into the server (no setup on Render)

Every **10 minutes** the backend calls its own public address `https://<your-service>.onrender.com/api/health`. The request goes out to the internet and back in through Render, so Render sees a visitor and keeps the service running.

- On Render it switches on by itself, because Render tells the app its public address (`RENDER_EXTERNAL_URL`).
- On other hosts, set `APP_KEEP_ALIVE_URL` to your backend's public URL.
- Change the interval with `APP_KEEP_ALIVE_INTERVAL` (e.g. `14m`). Keep it under 15 minutes.
- Turn it off with `APP_KEEP_ALIVE_ENABLED=false`.

Check it: Render **Logs** show `Keep-alive: pinging … every 10 min`. Then open `/api/health` a few times over an hour: `"uptime"` keeps growing (e.g. `2h 15m`) instead of resetting. ⚙️ Settings → **About** shows the same.

C.2 Layer 2: a scheduled GitHub event (also wakes it if it ever sleeps)

The repo includes `.github/workflows/keep-alive.yml`, a GitHub Actions **schedule** event that runs **every 10 minutes** and calls your backend's health address. Unlike layer 1, it can also wake the backend after a restart or a sleep.

1. GitHub → your repo → **Settings → Secrets and variables → Actions → Variables** tab → **New repository variable**.
2. Name: `BACKEND_URL` · Value: `https://<your-service>.onrender.com` → **Add variable**.
3. **Actions** tab → if asked, click **I understand my workflows, go ahead and enable them**.

4. Open **Keep backend awake → Run workflow** to test it now. The run summary shows `Backend answered: {"status":"UP",…}`.

From then on it runs by itself. Good to know:

- GitHub starts scheduled runs on a best-effort basis: they can be a few minutes late. Layer 1 covers the gaps.
- **Public repo:** free and unlimited. GitHub pauses schedules after **60 days with no commits**: you'll get an email, and one click (Actions → Keep backend awake → **Enable workflow**) restarts it.
- **Private repo:** Actions minutes are limited (2,000 per month on the free plan), and this schedule would use about 4,300. So the job **skips itself in private repos** and layer 1 still keeps the app awake. To force it on anyway, add a variable `KEEP_ALIVE_IN_PRIVATE_REPO` = `true`.

C.3 Optional layer 3: an external monitor

cron-job.org (free) → **Create cronjob** → URL `https://<your-service>.onrender.com/api/health` → every **10 minutes** → Save. UptimeRobot (free, 5-minute checks) also works and emails you if the app goes down.

C.4 Limits to know

- Render's free plan includes **750 instance hours per month**. One service running all month uses about 744, so **keep only one free service awake**. Otherwise they run out before the month ends and Render stops them until the next month.
- The health check **doesn't use the database**, so Neon can still sleep when nobody is chatting (saving its free compute hours). Neon wakes up in well under a second when someone sends a message.

Part A — New installation

1. How the free hosting fits together

```
   Phone / browser ──HTTPS──▶ Vercel  (React PWA: static files + service worker)
        │
        ├──HTTPS (REST) ─▶ Render  (Spring Boot API, Docker) ──▶ Neon (PostgreSQL)
        ├──WSS (live chat)▶ Render                              └─▶ Cloudinary (photos & videos)
        └◀── push notifications ── browser push service (Google / Mozilla / Apple / Microsoft) ◀──
Render
```

Piece	Service	Free plan highlights*
React frontend (PWA)	**Vercel** – Hobby	Global CDN, HTTPS, auto-deploy on every `git push`
Java API + WebSockets + push	**Render** – Free web service	Docker, HTTPS, auto-deploy; sleeps after ~15 min idle, wakes in ~1 min
Database	**Neon** – Free	Serverless Postgres, data persists
Photos & videos	**Cloudinary** – Free	No credit card, monthly credits for storage + bandwidth, global CDN
Push notifications	Built into browsers	Free; the server signs messages with its own VAPID keys (no Firebase account needed)

*Free-tier limits change from time to time, so check each provider's pricing page.

Why isn't the backend on Vercel too? Vercel serves static sites and short-lived serverless functions. It doesn't run Java, and those functions can't hold the WebSocket connections real-time chat needs. So the React app goes on Vercel and the Spring Boot API on Render.

Why Cloudinary? Render's free disk is temporary: files are wiped whenever the server restarts, which happens every time it wakes from sleep. Cloudinary keeps photos and videos permanently and serves them fast from a CDN.

---

2. Prerequisites

Accounts (sign up with GitHub or Google to save time):

- GitHub
- Vercel
- Render
- Neon
- Cloudinary

Tools (for running locally):

Tool	Version	Check with
Java JDK	17 or newer (21 recommended), e.g. Temurin	`java -version`
Node.js	20.19+ or 22+ (nodejs.org)	`node -v`
Git	any recent	`git --version`

You don't need Maven (the project includes `./mvnw`), a database (H2 is built in) or a Cloudinary account to run locally (media is saved to disk).

---

3. Run it locally

3.1 Get the code

Unzip the project (or clone your repository) and open a terminal in the `aurora-chat` folder.

3.2 Start the backend (port 8080)

```
cd backend
# macOS / Linux — APP_DEMO_DATA_ENABLED seeds demo users the first time
APP_DEMO_DATA_ENABLED=true ./mvnw spring-boot:run
```

Windows PowerShell:

```
cd backend
$env:APP_DEMO_DATA_ENABLED="true"; .\mvnw.cmd spring-boot:run
```

Windows Command Prompt:

```
cd backend
set "APP_DEMO_DATA_ENABLED=true" && mvnw.cmd spring-boot:run
```

The first run downloads dependencies (a few minutes). It's ready when you see `Started AuroraChatApplication`. Check http://localhost:8080/api/health, which should show `{"status":"UP",…}`.

> Local data lives in `backend/data/` (database + `uploads/` for photos and videos). Delete the folder to start fresh. Demo data is only added when the database is empty.

3.3 Start the frontend (port 5173)

Open a **second terminal**:

```
cd frontend
cp .env.example .env.local        # Windows: copy .env.example .env.local
npm install
npm run dev
```

Open http://localhost:5173.

3.4 Try it

- Sign in as **alex / password123**, or create your own account.
- Open a **private/incognito window** and sign in as **maya / password123**, then chat between the two windows.
- **Photos & videos:** click 🖼️ in the message box, paste an image, or drag files onto the chat. Click a photo to open the full-screen viewer.
- **Notifications:** open ⚙️ Settings (bottom-left) → turn on **Push notifications**, then **Send a test notification**. Browsers allow this on `http://localhost`. On other addresses you need HTTPS.
- **Phone layout:** open your browser's device toolbar (F12 → 📱) to see the mobile UI: long-press a message, swipe it right to reply, double-tap to ❤️.

> To try it on your real phone on the same Wi-Fi, use the deployed version (step 7 onward). Phones only allow notifications and installing on HTTPS sites.

3.5 Run the tests (optional)

```
cd backend && ./mvnw test         # 22 tests: chat, WebSocket, media, push, Cloudinary
cd frontend && npm run build      # strict TypeScript check + production build
```

3.6 Production-like run with PostgreSQL (optional, needs Docker)

```
docker compose up --build         # Postgres + API on http://localhost:8080
```

Then run the frontend as in step 3.3.

---

4. Push the code to GitHub

> Already have an Aurora Chat repo on GitHub? Don't create a new one: follow **Part B** to update it.

1. On GitHub click **New repository**, name it `aurora-chat`, and leave it **empty** (no README, .gitignore or license, because the project already has them).
2. In the project folder:

```
git init
git add .
git commit -m "Aurora Chat: Spring Boot + React real-time chat"
git branch -M main
git remote add origin https://github.com/<your-username>/aurora-chat.git
git push -u origin main
```

---

5. Create a free PostgreSQL database (Neon)

1. Go to https://console.neon.tech and click **New Project**.
2. Name it `aurora-chat` and pick a **region close to where you'll run the API**:
   - Render **Singapore** → Neon **AWS Asia Pacific (Singapore)** (good for India/Asia)
   - Render **Oregon** → Neon **AWS US West (Oregon)**
   - Render **Frankfurt** → Neon **AWS Europe (Frankfurt)**
3. Click **Create**, then **Connect**, and copy the **connection string**:

```
postgresql://neondb_owner:AbC123xyz@ep-cool-sky-123456.ap-southeast-1.aws.neon.tech/neondb?
sslmode=require&channel_binding=require
```

Paste it as-is later. The backend converts it to JDBC automatically and creates the tables on first start.

> 🔒 Treat connection strings and API secrets like passwords. Never commit them to Git.

---

6. Create free media storage (Cloudinary)

1. Sign up at https://cloudinary.com/users/register_free (no credit card).
2. Open the **Dashboard** (or **Settings → API Keys**).
3. Copy the **API environment variable**. It looks like:

```
CLOUDINARY_URL=cloudinary://123456789012345:AbCdEfGhIjKlMnOpQrStUvWxYz0@your-cloud-name
```

   You'll paste the part after `CLOUDINARY_URL=` into Render in the next step. Pasting the whole line also works.

That's all: the backend uploads files into an `aurora-chat` folder in your Cloudinary media library. When someone deletes a message, its files are deleted from Cloudinary too.

---

7. Deploy the Java backend (Render)

Use the included **Blueprint** (fastest) or create the service manually.

Option A — Blueprint (uses `render.yaml`)

1. Render dashboard → **New +** → **Blueprint**.
2. Connect GitHub if asked, then select the `aurora-chat` repository.
3. Render reads `render.yaml` and shows the `aurora-chat-api` service. Fill in:
   - `DATABASE_URL` → your Neon connection string
   - `CLOUDINARY_URL` → your Cloudinary URL (`cloudinary://…`)
   - `APP_CORS_ALLOWED_ORIGINS` → `https://*.vercel.app` for now (tightened in step 9)
   - `APP_PUSH_SUBJECT` → `mailto:` followed by your email, e.g. `mailto:you@example.com`
4. Click **Apply**. `APP_JWT_SECRET` is generated for you.

Option B — Manual web service

1. Render dashboard → **New +** → **Web Service** → select the `aurora-chat` repository.
2. Settings:

Field	Value
Name	`aurora-chat-api` (becomes part of your URL)
Region	same region as your Neon database
Branch	`main`
Root Directory	`backend`
Language / Runtime	**Docker** (auto-detected from `backend/Dockerfile`)
Instance Type	**Free**

3. **Environment Variables** → add:

Key	Value
`DATABASE_URL`	your Neon connection string
`CLOUDINARY_URL`	your Cloudinary URL
`APP_JWT_SECRET`	click **Generate** (or any long random string)
`APP_CORS_ALLOWED_ORIGINS`	`https://*.vercel.app` (tightened in step 9)
`APP_PUSH_SUBJECT`	`mailto:you@example.com`

4. **Advanced** → **Health Check Path** → `/api/health`.
5. Click **Create Web Service**.

Wait for the first deploy

The first Docker build takes about **5–10 minutes**. In the **Logs** tab you should see `Media storage: Cloudinary (cloud '…')` and `Started AuroraChatApplication`, and the service should show **Live**.

Copy your URL (e.g. `https://aurora-chat-api.onrender.com`) and open `…/api/health`. It should show `{"status":"UP",…}`. 🎉

> 🔑 **Push keys are automatic.** On first start the server generates its VAPID key pair and stores it in the database, so it survives restarts and redeploys. You don't need a Firebase account.
>
> 💤 On the free plan the service sleeps after ~15 minutes without traffic. The next visit wakes it in about a minute, and the app shows a "Waking up the server…" banner meanwhile. Push notifications are sent by the server, so a sleeping server sends nothing until someone opens the app (see step 11 to keep it awake).

---

8. Deploy the React frontend (Vercel)

1. Go to https://vercel.com/new and **import** your `aurora-chat` repository.
2. Configure the project:

Field	Value
Project Name	`aurora-chat` (your site becomes `aurora-chat.vercel.app` if available)
Framework Preset	**Vite** (auto-detected)
Root Directory	click **Edit** → choose **`frontend`** ⚠️ important
Build Command / Output	`npm run build` / `dist` (defaults; also in `vercel.json`)

3. **Environment Variables** → add `VITE_API_URL` = your Render URL, e.g. `https://aurora-chat-api.onrender.com` (**no trailing slash**).
4. Click **Deploy**. After about a minute you get a URL like `https://aurora-chat.vercel.app`.

`vercel.json` already sets the right caching for the service worker (`sw.js`) and the web app manifest, so installs and updates work.

> `VITE_*` variables are baked in at **build time**. If you change `VITE_API_URL`, redeploy: **Deployments** → ⋯ → **Redeploy**.

---

9. Connect frontend and backend (CORS)

1. Copy your Vercel production URL, e.g. `https://aurora-chat.vercel.app`.
2. Render → `aurora-chat-api` → **Environment** → edit `APP_CORS_ALLOWED_ORIGINS`:

```
https://aurora-chat.vercel.app
```

   To also allow preview deployments and a custom domain, separate them with commas:

```
https://aurora-chat.vercel.app,https://aurora-chat-*.vercel.app,https://chat.yourdomain.com
```

3. **Save Changes**. Render redeploys in a minute or two.

Rules: include `https://`, no trailing slash, no paths. The same list covers REST, file uploads and the WebSocket.

---

10. Test your live app, install it on your phone, turn on notifications

10.1 Chat

1. Open your Vercel URL. If the API was asleep you'll see "Waking up the server…" for up to a minute.
2. **Create an account**, then create a second one in a private window or on another device.
3. **New conversation** → pick the other user → chat. Try a photo, a video, a reply, a reaction and dark mode.

10.2 Download photos and videos you receive

Where	How
Computer	Hover over a photo or video → click the ⬇ **download** button in its corner. For a whole album: ⋯ → **Download all**. Files go to your *Downloads* folder as `aurora-<date>-<time>-xxxx.jpg`.
Phone	**Long-press** the message → **Save photo / Save video / Save all**. The share sheet opens: choose **Save Image / Save Video** (iPhone → Photos) or **Save / Download** (Android).
Full-screen viewer	Tap a photo → ⬇ (download) or **Share** (save to Photos, send to other apps).

10.3 Install as an app

Device	How
Android (Chrome/Edge/Samsung Internet)	Menu ⋮ → **Install app / Add to Home screen**, or ⚙️ Settings → **Install Aurora**
iPhone / iPad (Safari, iOS 16.4+)	Share button → **Add to Home Screen** → **Add**
Desktop (Chrome/Edge)	Install icon in the address bar, or ⚙️ Settings → **Install Aurora**

The installed app opens full-screen with its own icon, shows an unread badge on the icon, and loads even with a weak connection.

10.4 Turn on notifications

1. In the app, open ⚙️ **Settings** → switch on **Push notifications** → allow when the browser asks.
   - **iPhone/iPad:** this only works in the app opened **from the Home Screen** (an Apple requirement). Safari tabs don't offer it.
2. Tap **Send a test notification**. You should see "Notifications are working 🎉".
3. Close the app, then send yourself a message from your other account: a notification appears, and tapping it opens that chat.
4. To silence one chat: open it → tap the name at the top → **Mute notifications**.

Each device needs its own opt-in. Signing out stops notifications on that device.

---

11. Optional extras

Demo content in production

Set `APP_DEMO_DATA_ENABLED=true` on Render **before the first start against an empty database** to get the demo users. ⚠️ Anyone can sign in to them with `password123`, so turn it off again for a real community.

Keep the app awake

Built in and automatic on Render, plus a scheduled GitHub event every 10 minutes: see **Part C**.

Your own VAPID keys (optional)

The server generates and stores VAPID keys automatically. If you'd rather manage them yourself, run `npx web-push generate-vapid-keys` and set `APP_PUSH_VAPID_PUBLIC_KEY` and `APP_PUSH_VAPID_PRIVATE_KEY` on Render. Changing keys later means every device has to turn notifications off and on again.

Upload limits

Photos up to 10 MB and videos up to 50 MB by default (`APP_MEDIA_MAX_IMAGE_MB`, `APP_MEDIA_MAX_VIDEO_MB`). Photos are resized and compressed on the device first, so most uploads are much smaller. Cloudinary's free plan also caps the file size per upload, so keep videos short.

Custom domain

- Frontend: Vercel → Project → **Settings → Domains** → add `chat.yourdomain.com` and follow the DNS steps.
- Add the new origin to `APP_CORS_ALLOWED_ORIGINS` on Render.

Automatic deploys

Vercel and Render redeploy on every push to `main`. GitHub Actions run `.github/workflows/ci.yml` (builds and tests both apps on each push and pull request) and `.github/workflows/keep-alive.yml` (the 10-minute keep-awake event from Part C).

Other free backend hosts

`backend/Dockerfile` also runs on Koyeb, Google Cloud Run (free tier, billing account required) or an Oracle Cloud Always Free VM. Set the same environment variables. If the host has a **persistent** disk, you can skip Cloudinary (`APP_MEDIA_STORAGE=local`, `APP_MEDIA_DIR=/path/on/disk`).

12. Troubleshooting

Symptom	Cause & fix
Red banner **"Backend URL not configured"**	`VITE_API_URL` wasn't set when Vercel built the site. Add it and **redeploy**.
"Waking up the server…" never goes away	Open the API's `/api/health` directly. If it fails, check Render **Logs**: usually a typo in `DATABASE_URL` / `CLOUDINARY_URL` or a failed build.
Browser console: **CORS error**	`APP_CORS_ALLOWED_ORIGINS` doesn't exactly match your site (`https://your-app.vercel.app`, no trailing slash). Save and wait for the redeploy.

Symptom	Cause & fix
Sidebar stuck on **"Connecting…"**	The WebSocket can't connect: same CORS list, the API URL must be `https://` (it becomes `wss://`), and the API must be awake.
Photos/videos disappear after a while	`CLOUDINARY_URL` isn't set, so files went to Render's temporary disk. Render logs warn about this at startup. Set it (step 6) and re-share.
"Media unavailable" placeholders	Same as above (files lost), or the files were deleted from Cloudinary.
Upload says **"too large"**	Over the limit (10 MB photo / 50 MB video by default) or Cloudinary's free per-file cap. Trim the video or raise the limits.
HEIC photos rejected	Most browsers can't show HEIC. iPhones convert automatically when you pick from the photo library. Otherwise share as JPEG ("Most Compatible" camera setting).
A video **won't play**	Some codecs (e.g. HEVC from some phones) don't play in every browser. Re-record in "Most Compatible" mode or convert to H.264 MP4.
Notifications switch is greyed out	*Unsupported*: use Chrome, Edge, Firefox or Safari 16+. *Blocked*: allow notifications for the site in browser/OS settings. *iPhone*: open the app from the Home Screen.
No notifications arrive	Check: Settings → **Send a test notification**; the chat isn't muted; your phone isn't in Do Not Disturb/Focus mode; the Render service is awake. If the server's VAPID keys changed, turn notifications off and on again.
Notifications show while I'm in the app	They shouldn't: an open, focused app shows in-app banners instead. Check that the page is actually focused (not a background tab).
Old chats missing after upgrading	`DATABASE_URL` on Render must be the **same** Neon connection string as before. Don't create a new database. Check Render → Environment, fix it and redeploy; the data is still in Neon (and in your backup branch).
Everyone was **signed out after upgrading**	`APP_JWT_SECRET` changed. Data is fine: sign in again. To avoid it, keep the old value.
`update-repo.sh`: "uncommitted changes"	Your clone has local edits. Commit or discard them (`git stash`), or use a fresh `git clone`.
GitHub Desktop shows **thousands of changes**	You copied `node_modules` or `target`. Delete those folders, as they're ignored and regenerated automatically.
Server **still falls asleep**	Check `/api/health` → `"keepAlive":"on"`. In Render Logs, look for `Keep-alive: pinging …`. Set up Part C layer 2 or 3 as well. If Render suspended the service for the month, the free hours ran out (C.4).
Keep backend awake workflow never runs	Set the `BACKEND_URL` variable, enable workflows in the **Actions** tab, and make sure the file is on the default branch. In private repos it skips itself by design (C.2).
Download does nothing on iPhone	Use long-press → **Save photo**, then **Save Image** in the share sheet. In Safari tabs downloads go to the **Files** app → *Downloads*.
Downloads open the photo instead of saving	Old browser: right-click/long-press the opened photo → *Save image*.
Signed out after a redeploy	`APP_JWT_SECRET` changed or the database was reset. Sign in again and keep the secret stable.

Symptom	Cause & fix
All data disappears after each deploy	`DATABASE_URL` isn't set, so the API used the temporary H2 file. Set the Neon URL (step 5).
Render logs: `password authentication failed` / SSL errors	Re-copy the Neon string. URL-encode special characters in passwords (`@` → `%40`) and keep `sslmode=require` .
Vercel build fails	**Root Directory** must be `frontend` . Run `npm run build` locally to see the same error.
Port 8080 or 5173 already in use (local)	Stop the other process, or run the backend with `PORT=8081` and set `VITE_API_URL=http://localhost:8081` in `frontend/.env.local` .

5 · Verification results

Everything below ran against the real Spring Boot server and the production frontend build before delivery.

Upgrade test: old data survives (real PostgreSQL 17 like Neon, and the local H2 file database)

Step	What was checked	Result
1 · Old version (v1)	Demo data + a new user, a direct chat, messages, a reply, a reaction, an edit and read receipts (7 users, 6 chats, 21 messages, 15 reactions)	created
2 · Upgrade to v2.1	Same database, same secret: starts cleanly; users stay signed in; old passwords work; **every chat, message, reply, reaction, edit and read receipt identical**; row counts identical; only new tables + one nullable column added	**PASS**
3 · New features on old data	Photo upload and photo message in an old chat, mute, push keys generated	**PASS**
4 · Roll back to v1	Old version starts on the upgraded database, users still signed in, chats load, sending works	**PASS**
5 · Upgrade again	Messages from every version present; photo still attached and served	**PASS**
Repo update script	Run on a simulated old GitHub repo: pushes one commit, removes only the 5 obsolete screenshots, keeps the user's own files, local database and `.env`, keeps `mvnw` executable, and running it again changes nothing	**PASS**

Backend: `./mvnw test`, 26 tests

Suite	What it covers	Result
ChatFlowIntegrationTest (3)	Accounts, DMs, groups, replies, reactions, read markers, edit/delete rules, pagination	**PASS**
WebSocketIntegrationTest (3)	Real STOMP clients: message/typing/reaction push, presence, token and subscription rules	**PASS**
MediaIntegrationTest (2)	Photo/video upload with poster, range requests (206), disguised/oversized files refused, ownership, albums, captions, reply thumbnails, gallery, files deleted with the message, removal of uploads never sent	**PASS**
PushIntegrationTest (1)	Fake push service: encrypted + VAPID-signed delivery, decrypted payload, sender excluded, mute, test push, 410 removes the subscription, endpoint allowlist	**PASS**
WebPushCryptoTest (4)	Byte-for-byte match with Node's `http_ece` reference output, round trip, ES256 JWT, key encoding	**PASS**
Cloudinary · sniffer · DATABASE_URL (10)	Signature matches Cloudinary's documented example, signed multipart upload + delete, download URLs (`fl_attachment`), file-type detection, URL parsing	**PASS**
KeepAliveServiceTest (3)	Pings `/api/health` repeatedly on schedule, URL handling, idle when disabled or no URL	**PASS**

Browser end-to-end: phone (touch) + desktop, 40 checks

New in v2 (17)	Result
PWA manifest + service worker controls the page · offline reload shows the app shell	PASS
Phone layout: one pane, 16 px inputs, deep link `/c/:id`, back button returns to the list	PASS
Photo + video upload with progress; 3.5 MB photo compressed to 1.5 MB; video poster + duration; range 206	PASS
Album grid · desktop viewer (keys) · phone viewer (swipe next, swipe down to close)	PASS
Long-press action sheet + reaction · swipe-to-reply · double-tap ❤️ · reply to a photo shows a thumbnail	PASS
In-app banner for another chat (tap opens it) · mute silences a chat · shared-media gallery	PASS
Service worker push: shows notifications in the background, stacks per chat, stays quiet when focused	PASS
Settings sheet: theme switching, notification controls · no console errors	PASS
Downloads (7): hover ⬇ saves the photo; ⋯ → Download all (3 files); the video from the viewer is byte-identical; server fallback (`?download=true`) when the browser can't fetch; phone Save all; phone share sheet receives the image file	PASS
Keep-alive: the server pinged its own health endpoint on schedule (log + `"keepAlive":"on"`); the GitHub workflow passes schema validation, and its step succeeds, skips cleanly when not configured, and fails visibly when the backend is down	PASS
v1 regression (16): presence, typing, delivery (55 ms), replies, jump-to-quote, reactions, edit, delete, "Seen", groups, new DMs, theme	PASS
Push interop: Node's `http_ece` decrypts the server's real notification; the VAPID signature verifies	PASS

Production build: strict TypeScript + Vite, about 103 KB of gzipped JavaScript for the first load. The emoji picker sheet, media viewer, settings and new-chat dialog are split into separate chunks that load on demand.